

WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

1. Requirement Analysis

Product and Category Management

The e-commerce system needs to maintain information about products and their categories. Each product belongs to a particular category. The system stores important product details such as product name, price, and stock quantity.

The main objective of this module is to create and manage the **Category** and **Product** tables and establish a relationship between them.

Relationship

One category can contain many products. Therefore, the relationship between the Category and Product tables is **One-to-Many (1 → *)**.

- **Category_ID** is the Primary Key in the Category table.
- **Product_ID** is the Primary Key in the Product table.
- **Category_ID** acts as the Foreign Key in the Product table.

2. Category Table Creation

A Category table was created to store category information. The table contains **Category ID**, **Category Name**, and **Description**.

```
SQL> CREATE TABLE Category(  
2     Category_ID VARCHAR2(7),  
3     Category_Name VARCHAR2(20),  
4     Description VARCHAR2(50)  
5 );
```

Table created.

3. Product Table Creation

A Product table was created to store product information. It contains **Product ID, Product Name, Category ID, Price, and Stock Quantity**.

```
SQL> CREATE TABLE Product(  
 2     Product_ID VARCHAR2(7),  
 3     Product_Name VARCHAR2(20),  
 4     Category_ID VARCHAR2(7),  
 5     Price NUMBER(9,2),  
 6     Stock_Quantity NUMBER  
 7 );  
  
Table created.
```

4. Primary Key Constraints

Primary key constraints were added to both tables.

- Category_ID was defined as the Primary Key of the Category table.
- Product_ID was defined as the Primary Key of the Product table.

```
SQL> ALTER TABLE Category  
 2 ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);  
  
Table altered.  
  
SQL> ALTER TABLE Product  
 2 ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);  
  
Table altered.
```

5. Foreign Key Relationship

A foreign key relationship was established between the Product and Category tables. The Category_ID in the Product table references the Category_ID in the Category table.

This ensures that every product is associated with an existing category.

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);

Table altered.
```

6. NOT NULL Constraints

NOT NULL constraints were applied to the category name and product name. This ensures that these fields cannot contain empty values.

```
SQL> ALTER TABLE Category
  2  MODIFY Category_Name VARCHAR2(100) NOT NULL;

Table altered.

SQL> ALTER TABLE Product
  2  MODIFY Product_Name VARCHAR2(150) NOT NULL;

Table altered.
```

7. CHECK Constraints

CHECK constraints were added to validate product values.

- Product price must be greater than zero.
- Stock quantity must be zero or greater.

These constraints help maintain valid product data.

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Price CHECK (Price > 0);

Table altered.

SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.
```

8. UNIQUE Constraint

A UNIQUE constraint was applied to the Category Name so that duplicate category names cannot be entered.

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```

9. Insert Category Values

Eleven category records were inserted into the Category table for the Auto Hub automobile accessories and spare parts shop.

The categories include:

- Spare Parts
- Engine Oils
- Car Accessories
- Bike Accessories
- Helmets
- Batteries
- Seat Covers
- Floor Mats
- LED Lights
- Car Perfumes
- Mobile Holders

These categories represent the different types of automobile products handled by Auto Hub.

```
SQL> INSERT INTO Category VALUES ('Cat0021','Mobile Holders','Car and bike mobile holders');

1 row created.
```

10. Insert Product Values

Eleven product records were inserted into the Product table. The products include automobile spare parts, engine oils, accessories, helmets, batteries, seat covers, floor mats, LED lights, car perfumes, and mobile holders.

The inserted products are:

Product ID	Product Name	Category	Price	Stock Quantity
P2001	Brake Pad Set	Spare Parts	₹2,500	12
P2002	Castrol Engine Oil	Engine Oils	₹850	20
P2003	Car Phone Holder	Car Accessories	₹750	18
P2004	Bike Crash Guard	Bike Accessories	₹1,800	10
P2005	Steel Helmet	Helmets	₹1,500	14
P2006	Exide Car Battery	Batteries	₹6,500	7
P2007	Leather Seat Cover	Seat Covers	₹4,200	9
P2008	Premium Floor Mat	Floor Mats	₹2,200	11
P2009	LED Headlight	LED Lights	₹1,800	15
P2010	New Car Perfume	Car Perfumes	₹450	25
P2011	Mobile Holder	Mobile Holders	₹650	20

```
SQL> INSERT INTO Product VALUES ('P2011','Mobile Holder','Cat0021',650,20);  
1 row created.
```

11. Display Category and Product Records

The inserted records were displayed using the SELECT operation to verify that the data was successfully stored in the tables.

```
SQL> SELECT * FROM Category;
```

```
CATEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
Cat0011
```

```
Spare Parts
```

```
Car and bike spare parts
```

```
Cat0012
```

```
Engine Oils
```

```
Engine oils and lubricants
```

```
CATEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
Cat0013
```

```
Car Accessories
```

```
Car accessories
```

```
Cat0014
```

```
Bike Accessories
```

```
CATEGOR
```

```
-----
```

```
SQL> SELECT * FROM Product;
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
CATEGOR      PRICE STOCK_QUANTITY
```

```
-----
```

```
P2001
```

```
Brake Pad Set
```

```
Cat0011      2500          12
```

```
P2002
```

```
Castrol Engine Oil
```

```
Cat0012      850          20
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
CATEGOR      PRICE STOCK_QUANTITY
```

```
-----
```

```
P2003
```

```
Car Phone Holder
```

```
Cat0013      750          18
```

```
P2004
```

```
Bike Crash Guard
```

12. CRUD Operations

Create: A new product record was added to the Product table.

Read: The product records were retrieved and displayed from the Product table.

Update: The price of an existing product was modified.

Delete: A product record was removed from the Product table.

```
Connected.
```

```
SQL> INSERT INTO Product VALUES ('P2012','Steering Wheel Cover','Cat0013',900,10);
```

```
1 row created.
```

```
SQL> UPDATE Product SET Price=950 WHERE Product_ID='P2012';
```

```
1 row updated.
```

```
SQL> DELETE FROM Product WHERE Product_ID='P2012';
```

```
1 row deleted.
```

13. Category-wise Product Reports

Display All Products Under Each Category

The products available under each category were displayed along with their price and stock quantity.

Count Number of Products in Each Category

The number of products available in each category was calculated and displayed.

Find Highest-Priced Product in Each Category

The highest-priced product available in each category was identified.

Display Categories Having More Than 5 Products

The categories containing more than five products were identified using the category-wise product count.

Find Average Product Price Category-wise

The average product price for each category was calculated and displayed.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock_Quantity
  2  FROM Category c
  3  JOIN Product p ON c.Category_ID = p.Category_ID;
```

CATEGORY_NAME

PRODUCT_NAME

PRICE STOCK_QUANTITY

Spare Parts

Brake Pad Set

2500

12

Engine Oils

Castrol Engine Oil

850

20

CATEGORY_NAME

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Category c LEFT JOIN Product p ON c.Category_ID = p.Category_ID GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
-----
PRODUCT_COUNT
-----
Spare Parts
      1

Engine Oils
      1

Car Accessories
      1

CATEGORY_NAME
-----
PRODUCT_COUNT
-----
Bike Accessories
      1

Helmets
      1
```

```
SQL> SELECT c.Category_Name, MAX(p.Price) AS Highest_Price FROM Category c JOIN Product p ON c.Category_ID = p.Category_ID GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
-----
HIGHEST_PRICE
-----
Spare Parts
      2500

Engine Oils
      850

Car Accessories
      750

CATEGORY_NAME
-----
HIGHEST_PRICE
-----
```

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Category c JOIN Product p ON c.Category_ID = p.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID) > 5;
```

```
no rows selected
```

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Average_Price FROM Category c JOIN Product p ON c.Category_ID = p.Category_ID GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
-----
AVERAGE_PRICE
-----
Spare Parts
      2500

Engine Oils
      850

Car Accessories
      750

CATEGORY_NAME
-----
AVERAGE_PRICE
-----
Bike Accessories
      1800
```

implemented. The Category and Product tables were created, primary key and foreign key relationships were established, and constraints such as **NOT NULL**, **CHECK**, and **UNIQUE**

were applied. Category and product records were then inserted and used for CRUD operations and category-wise reporting.

This module provides the basic database structure required to manage products and their categories in an e-commerce system.